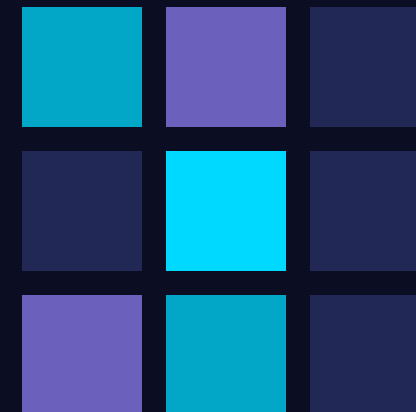


COMPUTAÇÃO GRÁFICA

Conceitos, Premissas e Fundamentos

Da matemática à imagem: como o computador gera, representa e exibe gráficos



Sumário

Introdução

Base Teórica: Fundamentos Matemáticos

Estrutura: Pipeline Gráfico

Modelagem: Representação de Objetos

Percepção: Cor e Percepção Visual

Renderização: Iluminação e Sombreamento

Renderização: Visibilidade e Superfícies Ocultas

Abordagens: Rasterização vs. Ray Tracing

Infraestrutura: Hardware e Arquitetura

Amostragem: Qualidade de Imagem

Aplicações: Áreas Relacionadas

Síntese/Conclusão

Definição e Escopo

Computação Gráfica é a área da ciência da computação dedicada à geração, manipulação, representação e exibição de imagens — estáticas ou dinâmicas — por meio de computadores.

Da cena ao pixel

Objetos matemáticos são transformados, iluminados e convertidos em uma grade de pixels coloridos exibida na tela.

Situa-se na interseção entre:



Matemática

Álgebra linear, geometria, cálculo



Física

Luz, ótica, dinâmica de corpos



Arte

Composição, cor, estética visual



Engenharia de Software

Algoritmos, hardware, APIs

Definição e Escopo

1. MATEMÁTICA: ÁLGEBRA LINEAR, GEOMETRIA, CÁLCULO

ÁLGEBRA LINEAR

Matriz A

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 0 & 3 & 4 \\ 2 & -1 & 5 \end{bmatrix}$$

Sistema linear $Ax = b$

$$\begin{cases} x + 2y - z = 1 \\ 3y + 4z = 7 \\ 2x - y + 5z = 3 \end{cases}$$

Vetores

$$\vec{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \vec{v} = \begin{bmatrix} 2 \\ -1 \\ 4 \end{bmatrix}$$

$$\vec{u} \cdot \vec{v} = 1 \cdot 2 + 2 \cdot (-1) + 3 \cdot 4 = 12$$

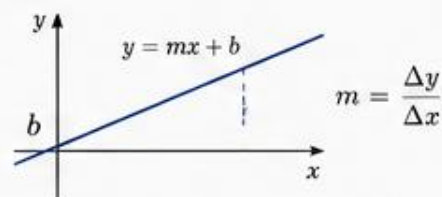
GEOMETRIA

Distância entre dois pontos

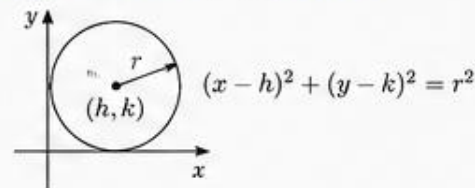
$$P(x_1, y_1) \quad Q(x_2, y_2)$$


$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Equação da reta



Círculo de centro (h, k) e raio r



CÁLCULO

Limite

$$\lim_{x \rightarrow a} f(x) = L$$

Derivada

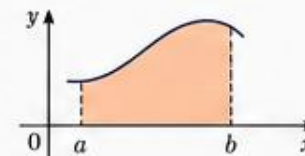
$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Exemplo: $\frac{d}{dx} (\sin x) = \cos x$

Integral

$$\int_a^b f(x) dx$$

representa a área sob a curva $y = f(x)$ no intervalo $[a, b]$.



Definição e Escopo

2. FÍSICA: LUZ, ÓTICA, DINÂMICA DE CORPOS

LUZ

A luz é uma onda eletromagnética.

Velocidade no vácuo:

$$c = 3,0 \times 10^8 \text{ m/s}$$

Intensidade luminosa

$$I = \frac{P}{4\pi r^2}$$

P : potência da fonte

r : distância da fonte

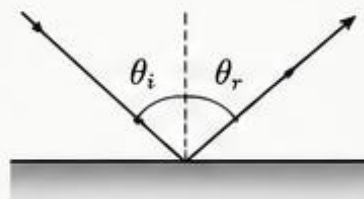
Espectro eletromagnético



ÓTICA

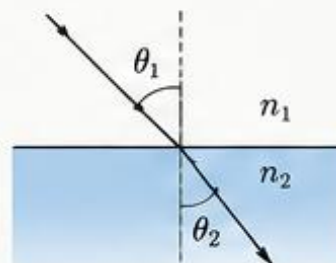
Lei da reflexão

$$\theta_i = \theta_r$$



Lei de Snell (refração)

$$n_1 \sin \theta_1 = n_2 \sin \theta_2$$



DINÂMICA DE CORPOS

Segunda Lei de Newton

$$\vec{F}_{res} = m\vec{a}$$

m : massa

$\vec{F}_{res}$: força resultante

$\vec{a}$: aceleração

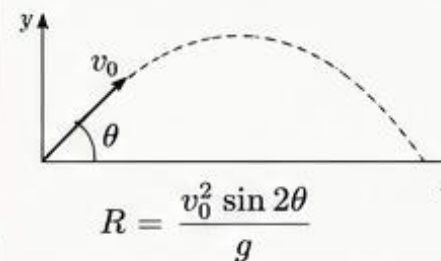
Força de atrito

$$f = \mu N$$

μ : coeficiente de atrito

N : força normal

Lançamento oblíquo



Definição e Escopo

3. ARTE: COMPOSIÇÃO, COR, ESTÉTICA VISUAL

COMPOSIÇÃO

Regra dos Terços



Linhas e Direção



Guiam o olhar e criam movimento e profundidade.

COR

Círculo Cromático



Harmonias de Cores

Complementares: 

Análogas: 

Triádicas: 

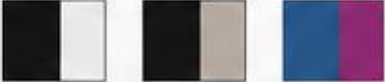
ESTÉTICA VISUAL

Equilíbrio



Distribuição visual que gera estabilidade.

Contraste



Destaque entre elementos cria interesse visual.

Unidade



Coerência entre os elementos transmite harmonia e propósito.

Definição e Escopo

4. ENGENHARIA DE SOFTWARE: ALGORITMOS, HARDWARE, APIS

ALGORITMOS

Busca Binária (exemplo)

```
function buscaBinaria(vetor,alvo):
    inicio = 0
    fim = len(vetor) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2
        if vetor[meio] == alvo:
            return meio
        elif vetor[meio] < alvo:
            inicio = meio + 1
        else:
            fim = meio - 1
    return -1
```

Complexidade

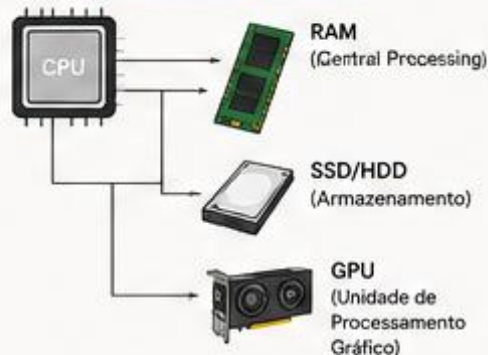
Busca Binária: $O(\log n)$

Busca Linear: $O(n)$

Ordenação QuickSort: $O(n \log n)$

HARDWARE

Componentes principais



Hierarquia de Memória



APIS

O que é uma API?

API (Application Programming Interface) é um conjunto de regras que permite a comunicação entre sistemas.

Exemplo de requisição HTTP

```
GET /api/produtos HTTP/1.1
Host: exemplo.com
Authorization: Bearer token123
Accept: application/json
```

Resposta (exemplo)

```
{
  "produtos": [
    {"id": 1, "nome": "Livro", "preco": 49.90},
    {"id": 2, "nome": "Caneta", "preco": 4.50}
  ]
}
```

Tipos de APIs

• REST (HTTP/JSON) • GraphQL • SOAP

Fundamentos Matemáticos

Toda imagem sintética nasce de operações sobre números

1 Álgebra Linear e Geometria Analítica

Vetores, matrizes, sistemas de coordenadas, produto escalar e vetorial.

2 Transformações Geométricas

Translação, rotação, escala e cisalhamento, compostas via matrizes homogêneas.

3 Geometria Projetiva

Projeções ortográficas e em perspectiva — do espaço 3D para a tela 2D.

4 Cálculo e Trigonometria

Curvas e superfícies (Bézier, B-splines), interpolação, iluminação.

5 Quatérnios e Números Complexos

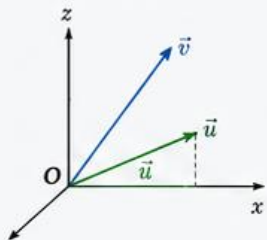
Representação estável de rotações, evitando o gimbal lock de Euler.

1. ÁLGEBRA LINEAR E GEOMETRIA ANALÍTICA

VETORES

$$\vec{v} = (x, y, z)$$

$$\vec{u} + \vec{v} = (x_u + x_v, y_u + y_v, z_u + z_v)$$



MATRIZES

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \quad C = A + B \quad D = AB$$

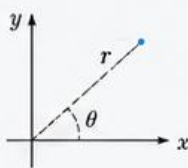
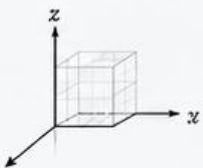
SISTEMAS LINEARES

$$Ax = b \quad \begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \end{cases}$$

SISTEMAS DE COORDENADAS

• Cartesiano (3D)

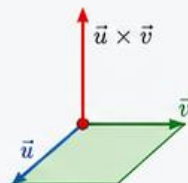
• Polar (2D)



PRODUTO ESCALAR E VETORIAL

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \|\vec{v}\| \cos \theta$$

$$\vec{u} \times \vec{v} = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix}$$

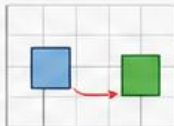


2. TRANSFORMAÇÕES GEOMÉTRICAS

TRANSFORMAÇÕES 2D

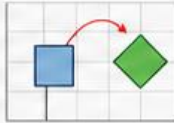
Translação

$$T(tx, ty) = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix}$$



Rotação (em torno da origem)

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



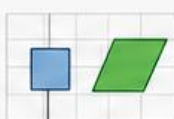
Escala

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Cisalhamento (shear)

$$H_x(k_x) = \begin{bmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

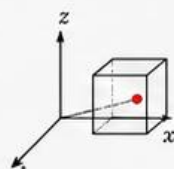


TRANSFORMAÇÕES 3D (homogêneas 4x4)

$$p' = Mp \quad \text{onde } p = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}, M \in \mathbb{R}^{4 \times 4}$$

Exemplo: Rotação em Z + Translação

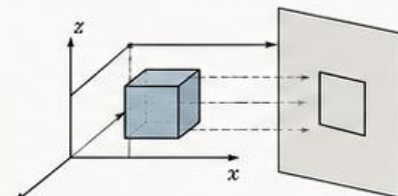
$$M = T(tx, ty, tz) R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & tx \\ \sin \theta & \cos \theta & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



3. GEOMETRIA PROJETIVA

PROJEÇÃO ORTOGRÁFICA

Paralelas ao eixo de projeção.



Matriz de projeção ortográfica (exemplo)

$$P_{ortho} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (\text{descarta } z)$$

PROJEÇÃO EM PERSPECTIVA

Linhas de projeção convergem para o centro de projeção (COP).



Matriz de projeção perspectiva (frustum simples)

$$P_{persp} = \begin{bmatrix} \frac{f}{a} & 0 & 0 & 0 \\ 0 & \frac{f}{b} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

onde f : distância focal, n : near, f : far, a : aspecto (right), b : aspecto (top)

ESPAÇO 3D → PROJEÇÃO → NORMALIZAÇÃO HOMOGÊNEA → VIEWPORT (TELA 2D)

Pipeline Gráfico Clássico

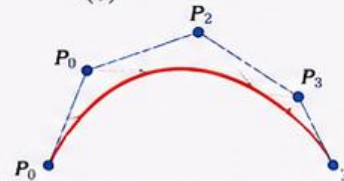
4. CÁLCULO E TRIGONOMETRIA

CURVAS DE BÉZIER

Pontos de controle $P_0, P_1, \dots, P_n$

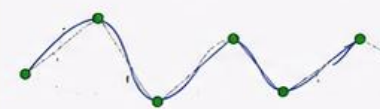
$$B(t) = \sum_{i=0}^n B_i^n(t) P_i, \quad t \in [0, 1]$$

$$B_i^n(t) = \binom{n}{i} (1-t)^{n-i} t^i$$



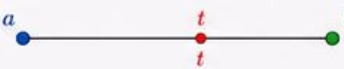
B-SPLINES

Curvas por partes com continuidade suave.



INTERPOLAÇÃO

$$\text{Lerp}(a, b, t) = (1-t)a + tb$$



ILUMINAÇÃO (Modelo de Phong)

$$I = k_a I_a + k_d I_d (\vec{N} \cdot \vec{L}) + k_s I_s (\vec{R} \cdot \vec{V})^\alpha$$

$\vec{N}$: normal da superfície

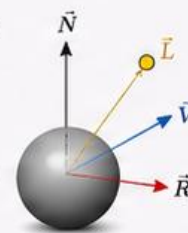
$\vec{L}$: direção da luz

$\vec{V}$: direção da câmera

$\vec{R}$: reflexão de $\vec{L}$

k_a, k_d, k_s : coeficientes

α : expoente de brilho



5. QUATÉRNIOS E NÚMEROS COMPLEXOS

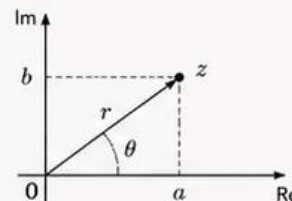
NÚMEROS COMPLEXOS

$$z = a + bi$$

$$|z| = \sqrt{a^2 + b^2}$$

$$z_1 z_2 = (a_1 + bi_1)(a_2 + bi_2)$$

Forma polar: $z = r(\cos \theta + i \sin \theta)$



QUATÉRNIOS

$$q = w + xi + yj + zk$$

$$q^* = w - xi - yj - zk \quad (\text{conjugado})$$

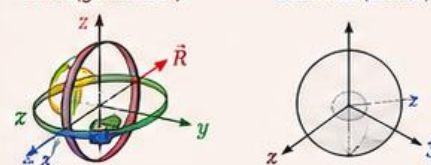
$$\|q\| = \sqrt{w^2 + x^2 + y^2 + z^2}$$

Rotação de um vetor $\vec{v}$:

$$\vec{v}' = q\vec{v}q^* \quad (\text{usando quatérnios puros})$$

Euler (gimbal lock)

Quatérnio (estável)



Conversão ângulo-eixo → quatérnio

$$q = \left[\cos \frac{\theta}{2}, \sin \frac{\theta}{2} (x, y, z) \right]$$

Vantagens:

- Interpolação suave (SLERP)
- Sem singularidades (gimbal lock)
- Eficiência em computação gráfica e robótica

Pipeline Gráfico

Sequência de etapas que transforma uma cena 3D em imagem final

1

Modelagem

Definição dos
objetos (malhas,
curvas)

2

Transformações

Posicionamento no
espaço e na câmera

3

Clipping

Recorte fora do
campo de visão

4

Rasterização

Conversão de
primitivas em pixels

5

Shading

Cálculo de cor de
cada fragmento

6

Exibição

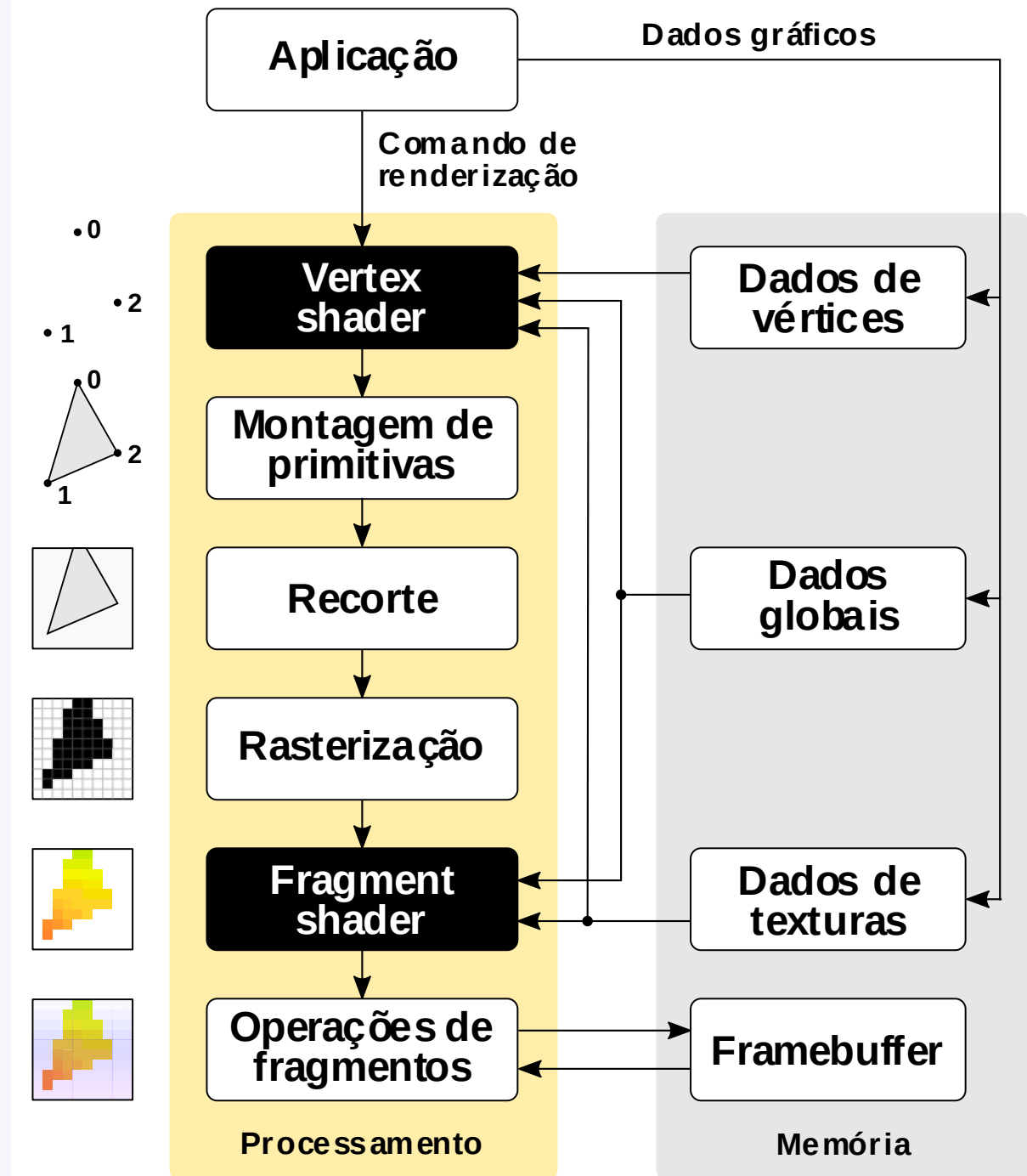
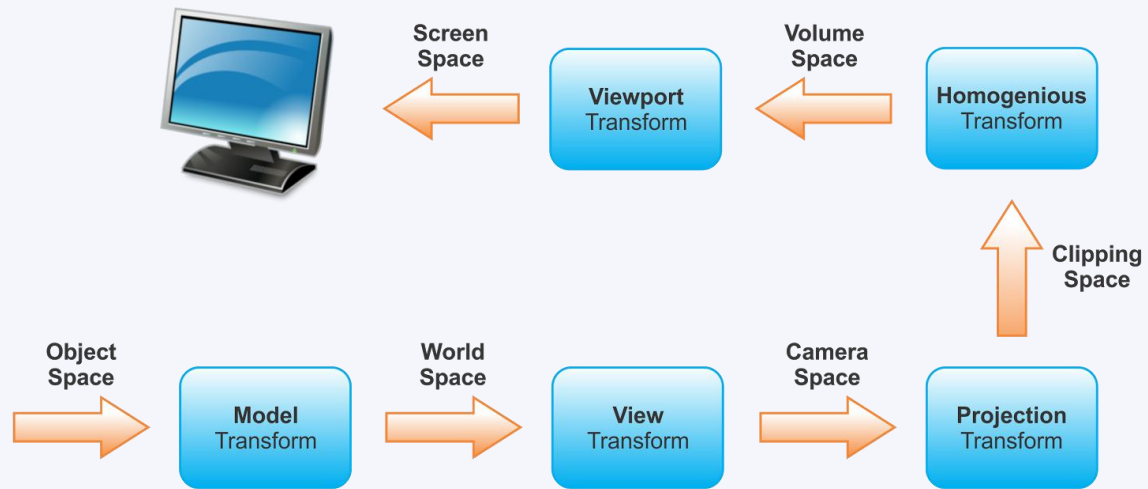
Composição final
no framebuffer

Etapas 1–3 preparam a geometria; 4–6 produzem a imagem em si (rasterização em destaque)

ESTRUTURA

Pipeline Gráfico

Sequência de etapas que transforma uma cena 3D em imagem final



Representação de Objetos



Malhas Poligonais

Representação mais comum; geralmente formadas por triângulos conectados.



Representações Paramétricas

Curvas e superfícies de Bézier, B-splines e NURBS.



Representações Volumétricas

Voxels — usados em visualização científica e médica.



CSG (Constructive Solid Geometry)

Combinação de sólidos primitivos via operações booleanas.

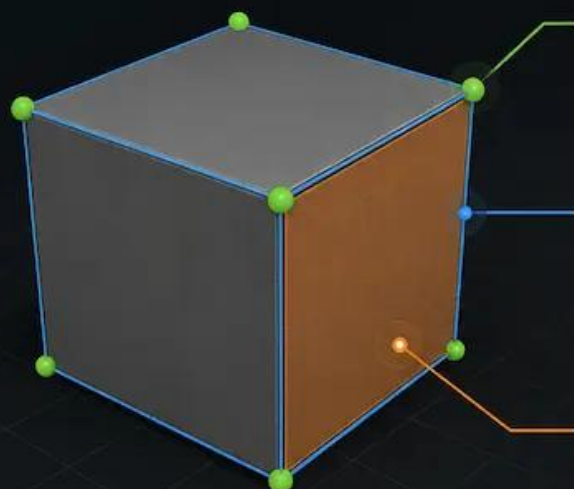


Malhas Poligonais

Representação mais comum; geralmente formadas por triângulos conectados.

VÉRTICES, ARESTAS E FACES EM MODELO 3D

Todo modelo 3D é construído a partir da combinação de três elementos fundamentais: vértices, arestas e faces (polígonos).



VÉRTICE

Ponto no espaço 3D. Define os cantos e extremidades do modelo.

ARESTA

Linha que conecta dois vértices. Define os contornos e a estrutura do modelo.

FACE

Superfície plana formada por três ou mais arestas. Define as superfícies do modelo.

TIPOS DE FACES (POLÍGONOS)



TRIÂNGULO
3 lados



QUADRILÁTERO (QUAD)
4 lados



PENTÁGONO
5 lados



HEXÁGONO
6 lados



N-GON (NGONS)
Mais de 4 lados



DICA: Quads (4 lados) são os mais utilizados em modelagem para **hard surface** e orgânica, pois oferecem um fluxo de arestas mais limpo e previsível para deformações.

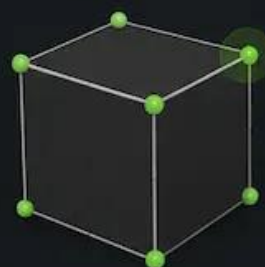


RESUMO:

VÉRTICES são os pontos, **ARESTAS** são as conexões e **FACES** são as superfícies. Juntos, eles formam qualquer objeto 3D que vemos em jogos, filmes, animações e muito mais!

VÉRTICES

- Representados por pontos.
- Possuem posição no espaço (coordenadas X, Y, Z).
- São os "pontos de controle" do modelo.

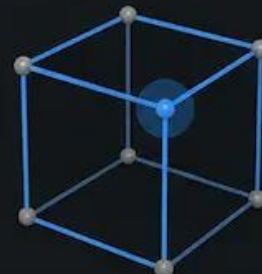


EXEMPLO:

Este cubo possui 8 vértices.

ARESTAS

- Conectam dois vértices.
- Formam o esqueleto (wireframe) do modelo.
- Definem a direção e os limites das faces.

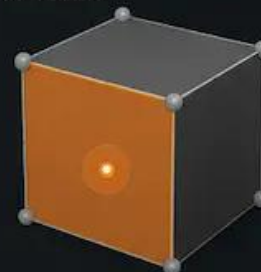


EXEMPLO:

Este cubo possui 12 arestas.

FACES

- Formadas por três ou mais arestas.
- Podem ter qualquer número de lados (triângulos, quads, ngons).
- Definem as superfícies visíveis do modelo.



EXEMPLO:

Este cubo possui 6 faces (quads).

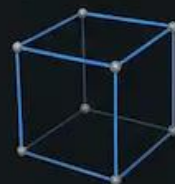
COMO UM MODELO 3D É CONSTRUÍDO

1 VÉRTICES



Primeiro definimos os pontos no espaço.

2 ARESTAS



Conectamos os pontos para criar a estrutura.

3 FACES



Preenchendo as áreas, criamos as superfícies.

4 MODELO FINAL



O modelo está pronto para texturizar, iluminar e renderizar.





Malhas Poligonais

Representação mais comum; geralmente formadas por triângulos conectados.

MALHA POLIGONAL EM MODELAGEM 3D

A malha poligonal é a estrutura fundamental dos modelos 3D. Ela é formada por **vértices**, **arestas** e **faces** (polígonos) que definem a forma e a superfície de um objeto.

COMPONENTES DA MALHA

VÉRTICE

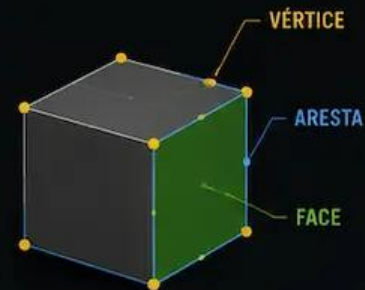
Ponto no espaço 3D (coordenadas X, Y, Z).

ARESTA

Linha que conecta dois vértices.

FACE (POLÍGONO)

Área plana formada por três ou mais arestas.



TIPOS DE POLÍGONOS



TRIÂNGULO
(3 lados)



QUADRILÁTERO
(4 lados)



PENTÁGONO
(5 lados)



HEXÁGONO
(6 lados)



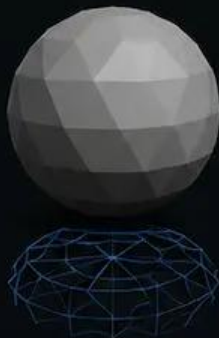
N-GON
(qualquer nº de lados)

Na prática, usamos principalmente **triângulos** e **quadriláteros**, pois são mais estáveis e compatíveis com a maioria dos motores e softwares 3D.

EXEMPLOS DE MALHA

BAIXA POLIGONAL (LOW POLY)

Menor quantidade de polígonos. Ideal para jogos, apps e projetos que exigem desempenho.



ALTA POLIGONAL (HIGH POLY)

Meior quantidade de polígonos. Ideal para close-ups, filmes, 3D para impressão, etc.



TOPOLOGIA

Organização da malha que afeta a deformação, animação e fluxo de arestas.



FLUXO DE ARESTAS (EDGE FLOW)

Um bom fluxo de arestas garante deformações limpas e previsíveis durante a animação.



DENSIDADE DA MALHA

Mais polígonos = mais detalhes, mas maior custo de processamento.



BAIXA

MÉDIA

ALTA

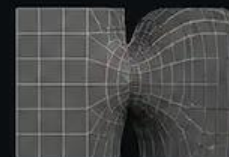
DESEMPENHO

DETALHES

FORMAS DE POLÍGONOS

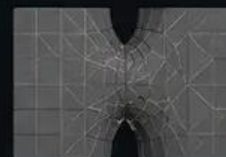
BOA TOPOLOGIA

Quads e tris bem distribuídos. Deformação suave.



TOPOLOGIA PROBLEMÁTICA

N-gons, triângulos longos e pólos mal posicionados. Deformação com artefatos.

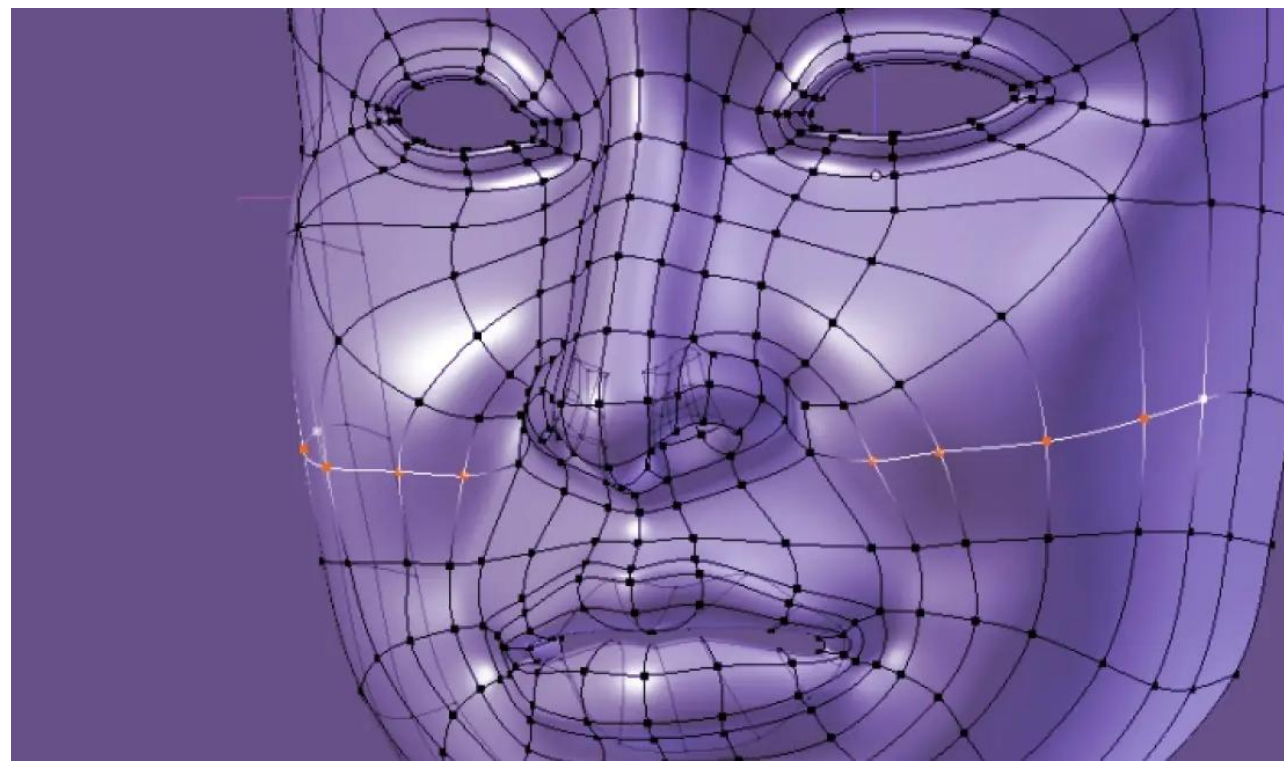
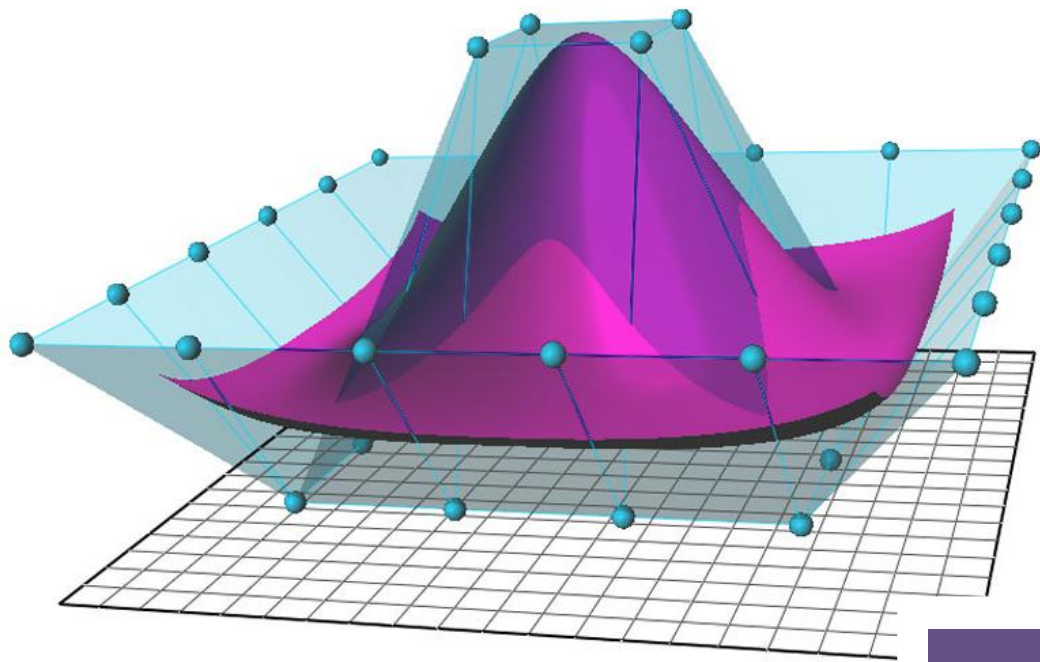


ONDE É USADA

- Jogos e Real-Time
- Animações e Filmes
- Arquitetura e Design
- Impressão 3D
- Produtos e Visualização



DICA: Sempre modele com uma topologia limpa desde o início. Isso facilita a animação, o texturização e o desempenho final do projeto.



Representações Paramétricas

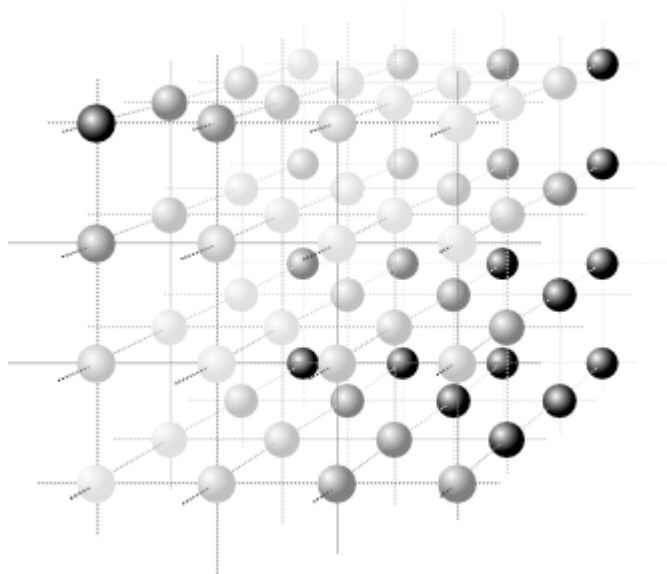
Curvas e superfícies de Bézier, B-splines e NURBS.



Representações Volumétricas

Voxels — usados em visualização científica e médica.

Ilustração de um gride de voxels, cada um contendo seu valor.



Um voxel destacado dentre outros em uma série de voxels empilhados.

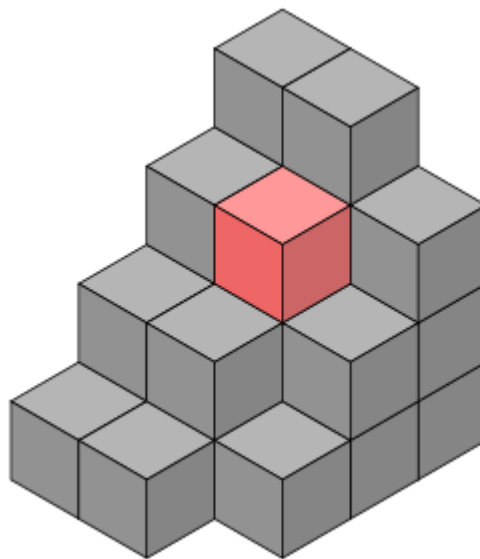
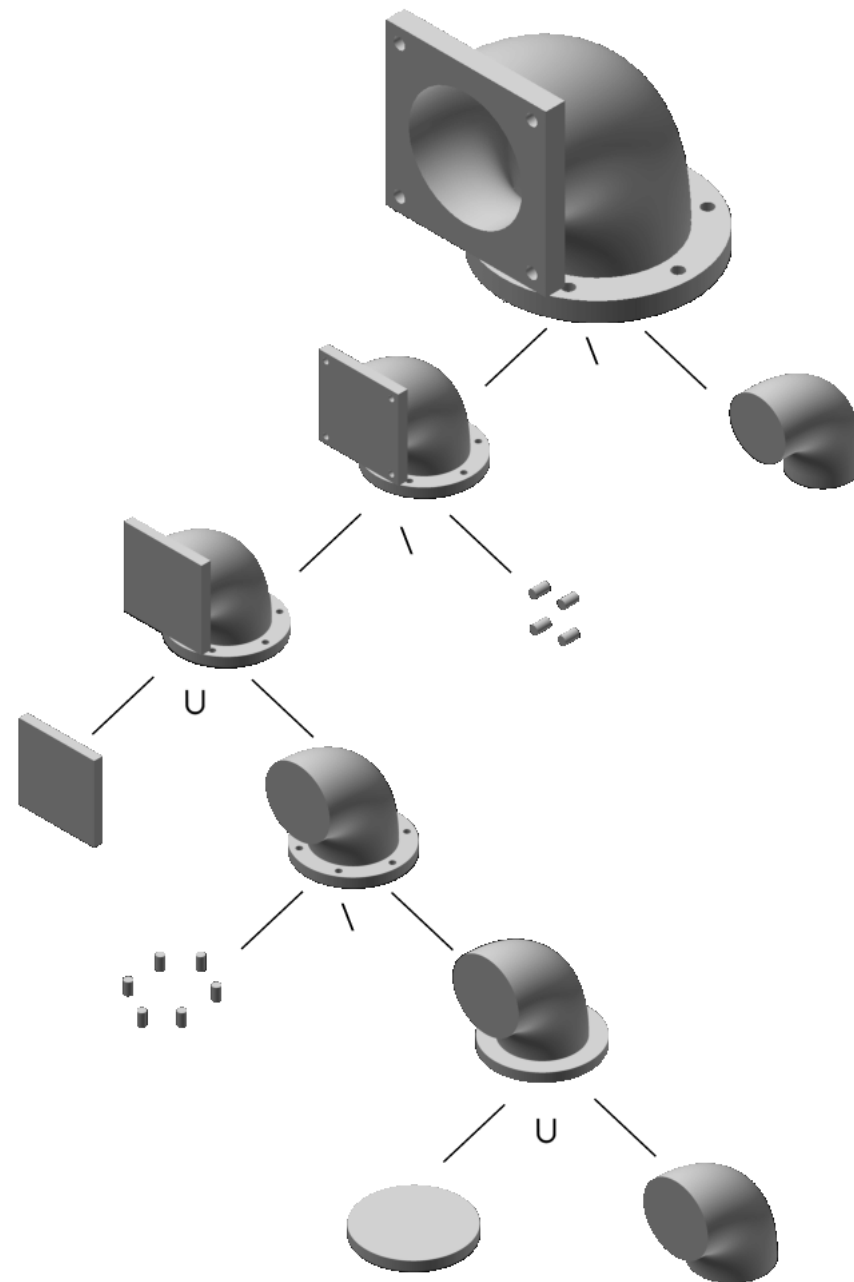
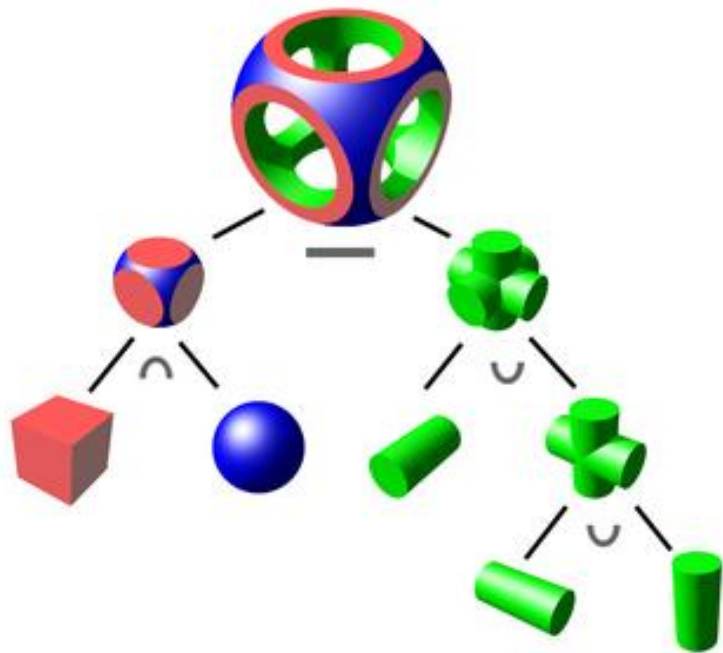


Imagem de voxel da superfície do Ribossomo criada usando o PyMOL.





CSG (Constructive Solid Geometry)

Combinação de sólidos primitivos via operações booleanas.

Cor e Percepção Visual

Modelos de Cor

RGB

Exibição em telas (aditivo)

CMYK

Impressão (subtrativo)

HSV / HSL

Edição intuitiva de cor

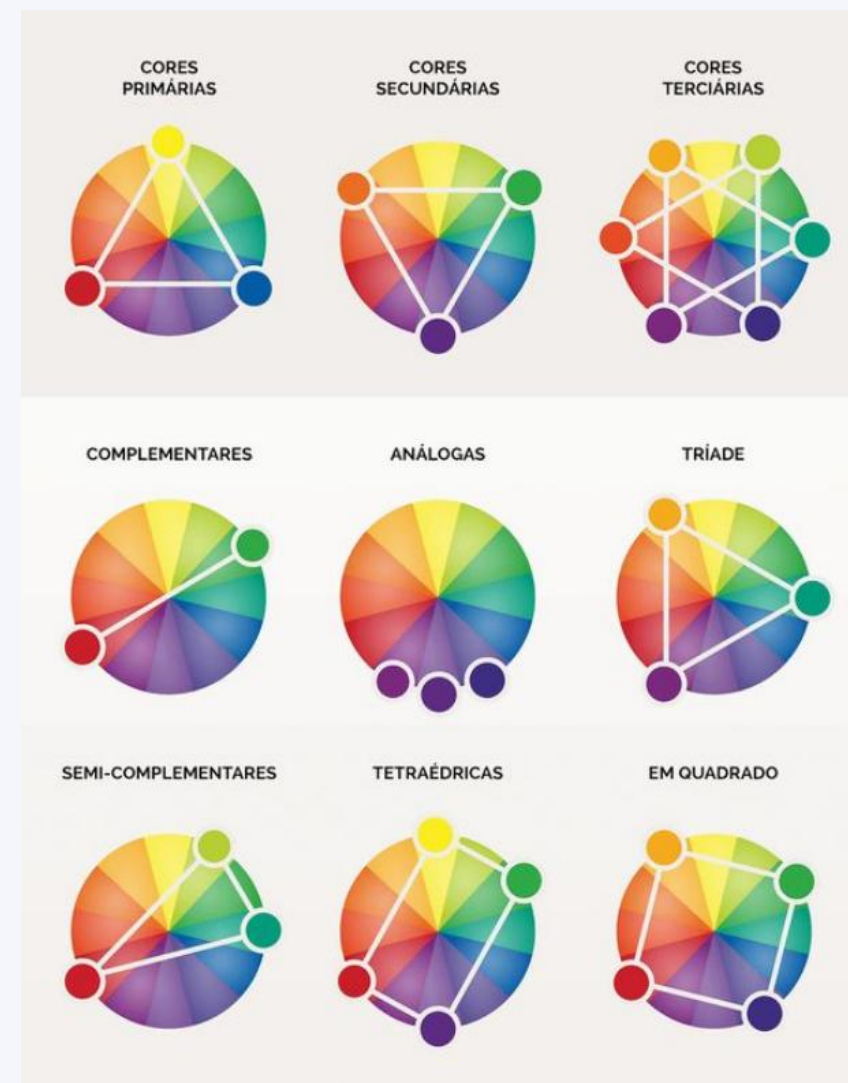
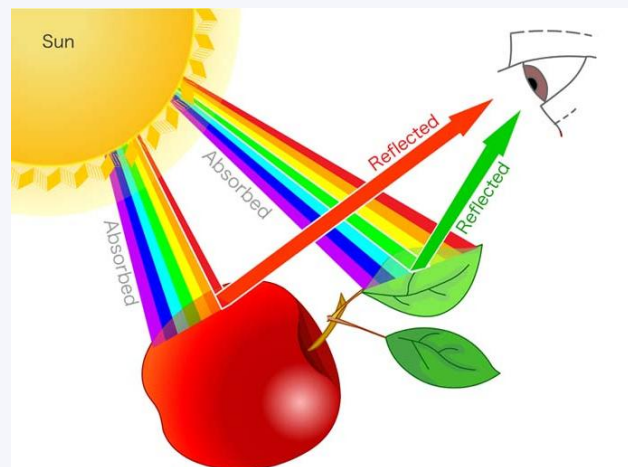
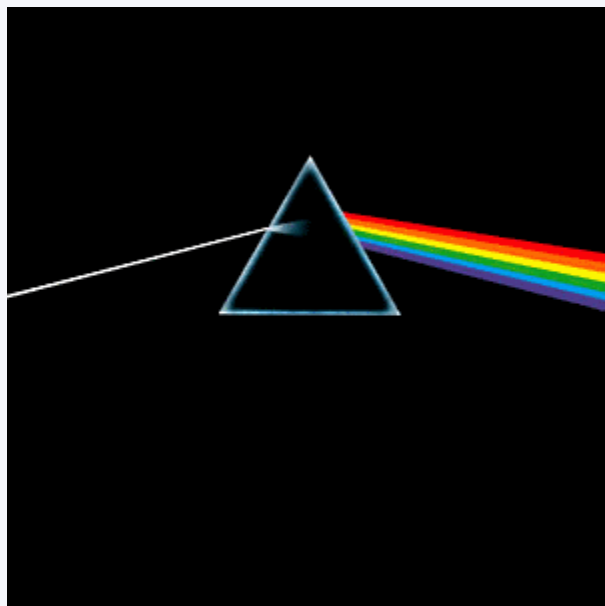
YUV

Compressão de vídeo

Percepção Humana e Correção Gamma

O olho e o cérebro não percebem luz de forma linear — essa base perceptual orienta técnicas como anti-aliasing e compressão de imagem. A correção gamma ajusta valores de cor para que a exibição em monitores corresponda à intensidade luminosa real.

Cor e Percepção Visual



Iluminação e Sombreamento

Iluminação Local

Modelo de Phong

Combina componentes ambiente, difusa e especular.

Blinn-Phong

Variação mais eficiente, muito usada em tempo real.

Textura

Texture, bump, normal e displacement mapping aplicam detalhe visual sobre superfícies.

Iluminação Global

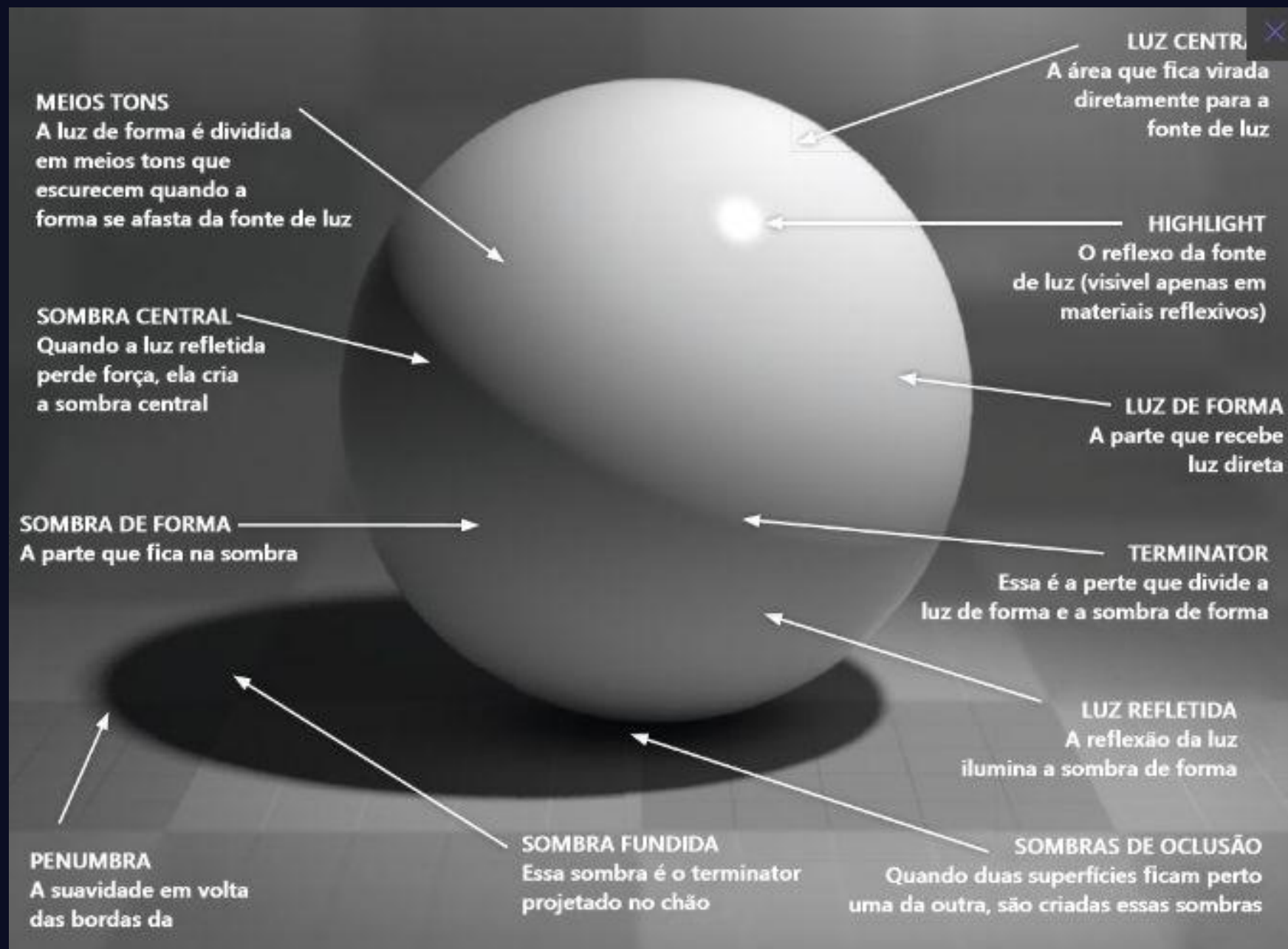
Simulam interações indiretas da luz — reflexões, refrações e sombras suaves.

Ray Tracing

Path Tracing

Radiosity

Iluminação e Sombreamento



Visibilidade e Superfícies Ocultas

Como determinar o que realmente aparece na imagem final

Z-Buffer (Depth Buffer)

Técnica fundamental e mais usada: armazena, para cada pixel, a distância (profundidade) da superfície mais próxima da câmera, garantindo que apenas a superfície visível seja desenhada.

Padrão em GPUs modernas por seu equilíbrio entre simplicidade e desempenho.

Back-face Culling

Descarta faces voltadas para longe da câmera antes mesmo de processá-las.

Painter's Algorithm

Desenha superfícies da mais distante para a mais próxima, como camadas de tinta.

BSP Trees

Particiona o espaço em árvore binária para ordenar visibilidade com eficiência.

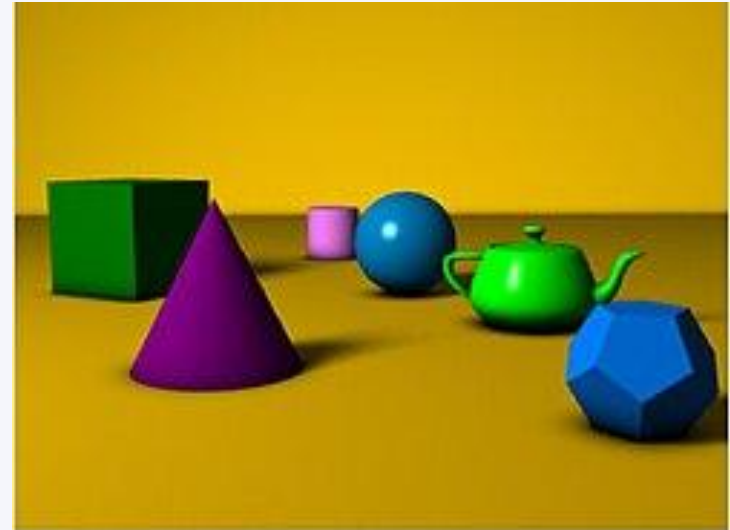
Visibilidade e Superfícies Ocultas

Como determinar o que realmente aparece na imagem final

Z-Buffer (Depth Buffer)

Técnica fundamental e mais usada: armazena, para cada pixel, a distância (profundidade) da superfície mais próxima da câmera, garantindo que apenas a superfície visível seja desenhada.

Padrão em GPUs modernas por seu equilíbrio entre simplicidade e desempenho.



A simple three dimensional scene



Z-buffer representation

Rasterização vs. Ray Tracing

As duas grandes estratégias para transformar geometria em pixels

RASTERIZAÇÃO

Projeta primitivas geométricas diretamente em pixels.

- Muito rápida
- Ideal para tempo real (jogos, GPUs)
- Menor precisão física de luz
- Base das APIs OpenGL/DirectX/Vulkan

RAY TRACING / PATH TRACING

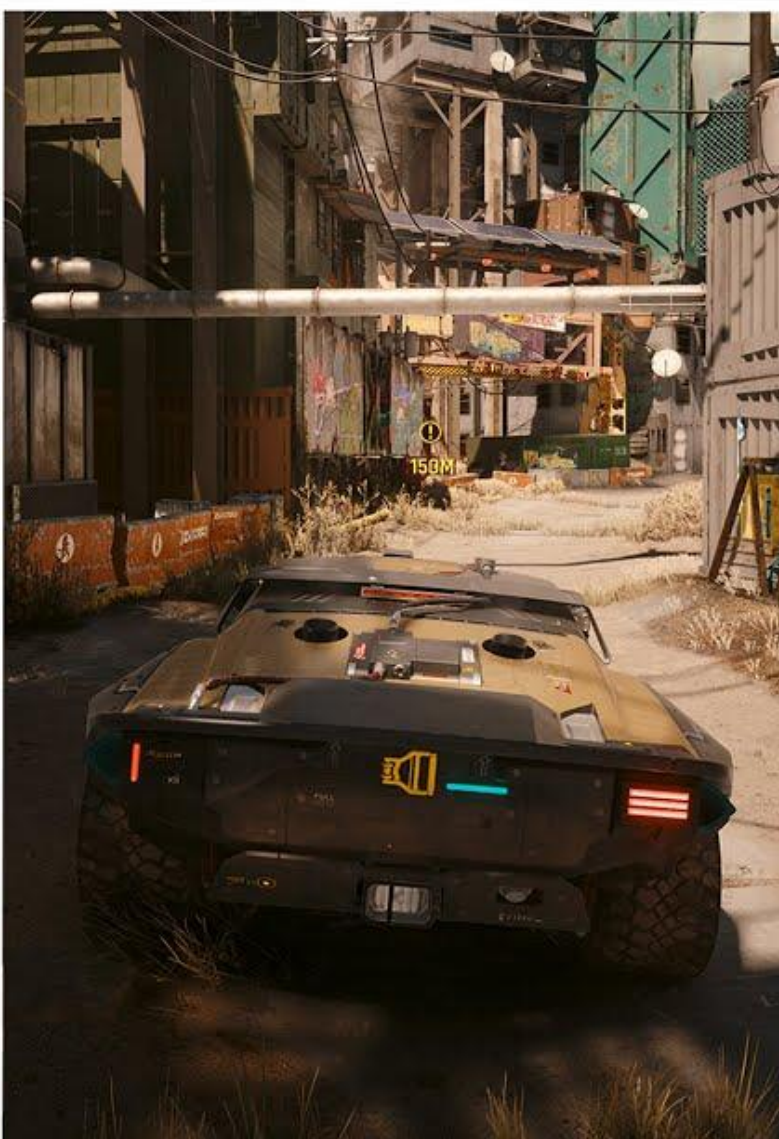
Traça raios de luz simulando seu comportamento físico real.

- Fisicamente mais precisa
- Historicamente offline (filmes, renders)
- Reflexos, refrações e sombras realistas
- Viável em tempo real com hardware dedicado (RTX)

RASTER

RAY TRACING

PATH TRACING



Hardware e Arquitetura

1

GPU

Graphics Processing Unit — arquitetura massivamente paralela, otimizada para operações vetoriais e matriciais.

2

APIs Gráficas

OpenGL, Vulkan, DirectX e Metal — interfaces padronizadas entre software e hardware gráfico.

3

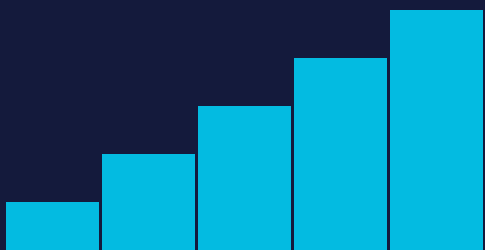
Shaders Programáveis

Vertex, fragment/pixel e compute shaders permitem customizar cada etapa do pipeline.

Qualidade de Imagem

Aliasing e Anti-aliasing

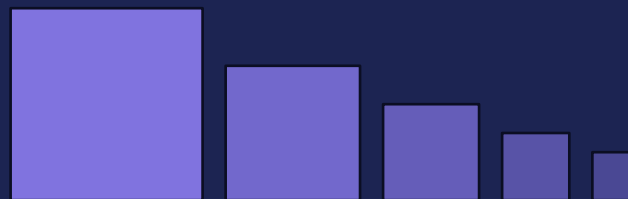
Surge da amostragem discreta de um sinal contínuo (imagem) — base no Teorema de Nyquist. Bordas serrilhadas (aliasing) são suavizadas por técnicas de anti-aliasing.



Bordas serrilhadas sem suavização

Filtragem e Mipmapping

Suaviza bordas e texturas. Mipmapping pré-calcula versões reduzidas de texturas para reduzir aliasing e melhorar desempenho conforme a distância da câmera.



Texturas em resoluções decrescentes

Áreas Relacionadas e Aplicadas



Animação Computacional

Keyframing, cinemática direta/inversa, simulação física.



Processamento de Imagens

Filtros, convolução, compressão — fronteira com visão computacional.



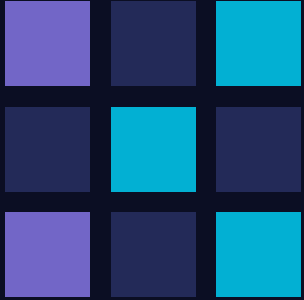
Realidade Virtual/Aumentada

Renderização estereoscópica e rastreamento espacial.



Visualização Científica

Aplicação dos princípios gráficos a dados abstratos e infográficos.



Em síntese

Computação Gráfica é matemática transformada em imagem

Da geometria e álgebra linear ao pipeline de renderização, da cor e iluminação ao hardware especializado — cada camada resolve um problema específico entre a cena imaginada e o pixel exibido na tela.

Obrigado!